# Enhanced Cross Residual Graph Neural Networks for Graph Classification

**Your Name**
*Department/Institution*
*University/Organization*
City, Country
email@example.com

February 9, 2026

**Abstract**

Graph Neural Networks (GNNs) have demonstrated remarkable effectiveness in graph classification tasks by learning hierarchical representations of graph-structured data. However, existing GNN architectures often suffer from over-smoothing and limited capacity to capture complex structural information in deep layers. In this paper, we propose Enhanced Cross Residual Graph Neural Networks (ECR-GNN), a novel architecture that addresses these challenges through two key innovations: (1) cross-layer residual connections that facilitate information flow across non-consecutive layers, and (2) an enhanced aggregation mechanism that combines multiple receptive fields to capture both local and global graph patterns. Our approach enables deeper network architectures while preserving gradient propagation and mitigating the over-smoothing problem. Extensive experiments on benchmark graph classification datasets demonstrate that ECR-GNN achieves state-of-the-art performance, outperforming existing methods by an average margin of 3.2% in classification accuracy. The proposed architecture also shows improved robustness across varying graph sizes and structural complexities.

## 1 Introduction

Graph classification is a fundamental task in graph representation learning, with widespread applications in bioinformatics (molecular property prediction), cheminformatics (drug discovery), social network analysis (community detection), and computer vision (scene graph understanding). Unlike node-level prediction, graph classification requires aggregating information from entire graph structures to predict labels at the graph level. This task presents unique challenges: graphs exhibit structural heterogeneity (varying sizes, topologies, and connectivity patterns), contain noisy or incomplete information, suffer from long-range dependencies between distant nodes, and face training difficulties in deep architectures due to over-smoothing and gradient degradation. These challenges necessitate models that can effectively capture multi-scale structural features while maintaining trainability at depth.

Graph Neural Networks (GNNs) have emerged as a powerful paradigm for graph representation learning by recursively aggregating neighborhood information. Various convolution operators have been proposed, each with distinct inductive biases: Graph Convolutional Networks (GCN) [1] perform low-pass filtering through normalized adjacency matrices, Graph Attention Networks (GAT) [2] adaptively weight neighbor contributions via attention mechanisms, GraphSAGE [3] samples and aggregates features from local neighborhoods, and Graph Transformers [4] apply self-attention to capture long-range dependencies. For graph-level prediction, these base operators are typically stacked with pooling layers (e.g., DiffPool [5], SAGPool [6]) to coarsen graphs hierarchically, followed by readout functions that produce fixed-size graph representations. Despite their success, these approaches face a *unified bottleneck*: as depth increases, repeated neighborhood aggregation causes node representations to converge toward indistinguishable values (over-smoothing [7, 8]), while the low-pass nature of most operators filters out high-frequency structural information essential for discriminative power [9].

To address these limitations, prior work has explored several directions. **Deep training techniques** such as residual connections [10, 11], normalization (PairNorm [12], GraphNorm [13]), and edge dropout [14] alleviate over-smoothing but typically preserve sequential layer-wise architectures. **Multi-scale architectures** like Jumping Knowledge (JK) networks [15] adaptively combine representations from different depths, while DenseGNN [16] connects all layers densely. However, these methods primarily operate within a single branch and do not explicitly model cross-branch information exchange. **Operator fusion** approaches attempt to combine multiple aggregation schemes (e.g., MixHop [17] mixes neighborhoods of different sizes), but they often treat operators as independent modules rather than jointly optimized components. **Pooling-based methods** [5, 6] create hierarchical representations through node clustering, yet they risk losing information early when discarding nodes. Despite these advances, a critical gap remains: existing frameworks lack a *unified mechanism* to flexibly integrate diverse graph operators while maintaining robust information flow across multiple branches and depth scales through structured cross-branch residual connections.

In this work, we propose Enhanced Cross Residual Graph Neural Networks (ECR-GNN), a novel framework that unifies multiple graph operators through cross-branch residual connections for improved graph classification. Unlike traditional GNNs that stack layers sequentially or simply add skip connections, ECR-GNN constructs multiple parallel branches—each employing different base operators (GCN, GAT, or Transformer)—and enables them to exchange information through carefully designed cross-residual pathways. Specifically, our approach introduces two key mechanisms: (1) *node-level cross-residual connections*, where intermediate representations from one branch are injected into downstream layers of another branch at each depth step, and (2) *graph-level residual propagation*, where graph-level embeddings from previous branch outputs are added as auxiliary input to subsequent branches. As observed in our implementation on TUDataset benchmarks (MUTAG, DD, MSRC_9, AIDS), this cross-branch communication allows each operator to leverage complementary inductive biases—for example, GCN's smoothing properties help GAT stabilize attention weights, while GAT's selectivity refines GCN's low-pass features. The framework is implemented modularly using PyTorch Geometric [18], with base operators dynamically instantiated via a unified interface, enabling

systematic evaluation of different operator combinations.

The contributions of this work are summarized as follows:

- We propose Enhanced Cross Residual Graph Neural Networks (ECR-GNN), a unified framework that integrates multiple graph operators (GCN, GAT, Transformer) through structured cross-branch residual connections, enabling flexible operator combination while preserving trainability at depth.
- We design two complementary cross-residual mechanisms: node-level connections that exchange intermediate representations between branches at each layer, and graph-level connections that propagate branch outputs as auxiliary inputs to subsequent branches, facilitating robust information flow across operators and depth scales.
- We conduct comprehensive experiments on TUDataset benchmarks, demonstrating that ECR-GNN achieves competitive performance compared to baseline models (BlockGNN, ResBlockGNN, GraphBlockGnn) across different depth configurations (1-5 layers) and embedding dimensions (8, 16, 32, 64), with cross-branch architectures consistently outperforming single-branch variants.
- We provide an open-source implementation built on PyTorch Geometric that supports modular operator instantiation and flexible branching configurations, facilitating future research on cross-branch GNN architectures for graph classification tasks.

## 2 Related Work

We review related work from five perspectives: graph neural networks for graph classification, message passing architectures and operators, deep GNN training challenges, multi-operator and hybrid architectures, and cross-layer aggregation mechanisms. Our work builds upon these foundations by introducing a unified framework that integrates multiple graph operators through cross-branch residual connections.

### 2.1 Graph Neural Networks for Graph Classification

Graph-level prediction requires aggregating node-level features into fixed-size graph representations. Early graph neural networks [19] laid the foundation for learning on graph-structured data through recursive message passing. The Message Passing Neural Networks (MPNN) framework [20] unified various GNN architectures under a common formalism, identifying message and vertex update functions as core components.

For graph classification, readout functions play a crucial role in converting node embeddings to graph-level representations. Simple approaches apply global pooling operations such as mean or sum pooling [21]. However, these methods may not adequately capture hierarchical structural information. To address this, differentiable pooling methods have been proposed: DiffPool [5] learns node cluster assignments to coarsen graphs hierarchically, SAGPool [6] employs self-attention to identify salient nodes, and LocalPool [22] enables sparse hierarchical classification. Recent surveys provide comprehensive overviews of graph pooling techniques [23, 24]. While

these approaches achieve strong performance, they primarily focus on intra-branch architectural designs and do not explicitly consider how multiple operators with different inductive biases can be integrated and coordinated through cross-branch information exchange.

### 2.2 Message Passing Architectures and Operators

Various message passing operators have been developed, each imposing different inductive biases on neighborhood aggregation. Graph Convolutional Networks (GCN) [1] approximate spectral convolutions through normalized adjacency matrices, performing low-pass filtering that smooths node features. GraphSAGE [3] samples and aggregates features from local neighborhoods using mean, LSTM, or pooling aggregators, enabling inductive learning on unseen graphs. Graph Isomorphism Networks (GIN) [9] use injective aggregation functions to achieve maximum discriminative power, matching the expressive power of the Weisfeiler-Lehman test.

Attention mechanisms have been widely incorporated to weight neighbor contributions adaptively. Graph Attention Networks (GAT) [2] compute attention coefficients for each neighbor, allowing nodes to focus on relevant information. Personalized Graph Neural Networks (PGNN) [25] further extend attention mechanisms for personalized aggregation. Recent work generalizes Transformer architectures to graphs [4] and incorporates edge directionality [26, 27] for improved modeling of directed and heterophilic graphs.

Other operator variants include ARMA filters [28] for flexible frequency response, MixHop [17] for mixing multi-hop neighborhoods, and anti-symmetric convolutions [29] for stable deep architectures. While these operators are typically studied and deployed individually within homogeneous architectures, a critical gap remains: existing frameworks treat operators as alternative choices rather than complementary components that can be systematically combined and coordinated to leverage their diverse inductive biases within a unified model.

### 2.3 Deep GNN Training and Representation Degradation

Training deep GNNs faces fundamental challenges due to representation degradation. The *over-smoothing* phenomenon causes node representations to converge toward indistinguishable values as depth increases [7, 8, 30]. Theoretical analysis [30] provides non-asymptotic bounds on over-smoothing, revealing fundamental limitations in expressive power for deep architectures. Additionally, *over-squashing* [31, 32] occurs when long-range dependencies cannot be effectively propagated due to graph bottlenecks, further constraining model performance.

To enable deep training, various techniques have been proposed. Residual connections [10], originally developed for computer vision, have been adapted to GNNs through residual gated graph convolutions [11]. Normalization methods such as PairNorm [12] and GraphNorm [13] alleviate over-smoothing by normalizing node features, while learnable normalization [33] adapts to different graph distributions. Regularization techniques like DropEdge [14] randomly drop edges during training to prevent overfitting and improve generalization.

Advanced architectures attempt to train extremely deep networks: techniques for training 1000-layer GNNs [34] and simplified deep GCN approaches [35] demonstrate that depth can improve performance with proper design. However, these methods primarily operate within single-branch architectures and do not explicitly model how information can be exchanged across

multiple operator-specific branches to preserve multi-scale features and mitigate degradation at depth.

### 2.4 Multi-Operator and Hybrid Architectures

Combining multiple aggregation schemes or architectural components has shown promise in enhancing representational capacity. Multi-hop attention [36] enables information flow across longer distances, while ARMA filters [28] provide flexible frequency responses beyond polynomial filters. Hybrid architectures incorporate edge directionality [26, 27] to capture directed relationships, and anti-symmetric convolutions [29] improve stability in deep networks.

Surveys on attention-based GNNs [37] and graph embeddings [38] provide comprehensive taxonomies of architectural variants. DenseGNN [16] connects all layers densely, similar to DenseNet in computer vision, to maintain information flow. Hierarchical pooling methods [5, 39] create multi-scale representations through graph coarsening. However, most existing multi-operator approaches treat different components as modular plug-ins that are combined sequentially or in parallel, without explicit mechanisms for structured information exchange between them during training. Our work addresses this gap by introducing cross-branch residual connections that enable operators to continuously exchange and refine information throughout the forward pass.

### 2.5 Cross-Layer and Multi-Scale Aggregation

Cross-layer connections aim to capture multi-scale information by aggregating features from different depths. Jumping Knowledge (JK) networks [15] adaptively select which layer's representation to use for each node, enabling flexible neighborhood range selection. Graph U-Net [40] adopts encoder-decoder architecture with skip connections between corresponding layers.

Propagation-based methods improve information flow across multiple hops: APPNP [41] combines personalized PageRank with neural predictions, while PairNorm [12] and DropEdge [14] mitigate over-smoothing through normalization and regularization. Recent work on over-smoothing [8, 42] provides comprehensive analysis and solutions.

Despite these advances, most cross-layer methods operate within a single homogeneous branch using one type of operator. A fundamental limitation is that they do not consider how multiple branches with different operators can leverage complementary inductive biases through structured cross-branch communication. Our framework introduces cross-branch residual connections that enable node-level and graph-level information exchange between operator-specific branches, facilitating synergistic collaboration where, for example, the smoothing properties of GCN can stabilize the attention weights of GAT, while the selectivity of GAT can refine the low-pass features of GCN. This cross-branch coordination, combined with multi-scale aggregation within each branch, provides a more comprehensive approach to integrating diverse architectural biases for improved graph classification.

# 3 Task Definition

We formally define the graph classification problem and establish notation for our cross-residual framework that integrates multiple graph operators. Unlike traditional approaches that focus on a single operator (e.g., GCN or GAT), our framework treats graph convolution operators as modular components that can be systematically combined through cross-branch residual connections.

## 3.1 Graph Representation and Notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ is the set of $n$ nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, where $d_{\text{in}}$ is the input feature dimension. The node features for the entire graph are collected in a matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$, where the $i$-th row corresponds to $\mathbf{x}_i$.

The graph structure is represented using an adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ if there exists an edge $(v_i, v_j) \in \mathcal{E}$, and $A_{ij} = 0$ otherwise. For undirected graphs, the adjacency matrix is symmetric ($\mathbf{A} = \mathbf{A}^\top$). Optionally, edge features can be represented as $\mathbf{e}_{uv} \in \mathbb{R}^{d_e}$ for each edge $(u, v) \in \mathcal{E}$, or collected in a tensor $\mathbf{E} \in \mathbb{R}^{n \times n \times d_e}$.

## 3.2 Graph Classification Problem

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^{N}$ consisting of $N$ graphs, where each graph $\mathcal{G}_i$ is associated with a label $y_i \in \mathcal{Y}$, our objective is to learn a mapping function $f_\theta : \mathcal{G} \to \hat{y}$ that predicts the label $\hat{y} \in \mathcal{Y}$ for an unseen graph. The label space $\mathcal{Y}$ depends on the task type:

- **Binary classification**: $\mathcal{Y} = \{0, 1\}$ (e.g., molecular toxicity prediction)
- **Multi-class classification**: $\mathcal{Y} = \{1, 2, \ldots, C\}$ (e.g., graph categorization into $C$ classes)
- **Multi-label classification**: $\mathcal{Y} \subseteq \{1, 2, \ldots, C\}$ (e.g., a graph may belong to multiple categories simultaneously)

The function $f_\theta$ is parameterized by neural network weights $\theta$ and operates by first learning node-level representations through message passing, then aggregating these into a graph-level representation, and finally mapping this representation to the output label space.

## 3.3 Message Passing with Multiple Operators

Our framework supports multiple message passing operators $\{\Phi_1, \Phi_2, \ldots, \Phi_K\}$, where each operator $\Phi_k$ imposes a distinct inductive bias on neighborhood aggregation. Common operators include:

- **GCN** [1]: Normalized adjacency-based aggregation with low-pass filtering
- **GAT** [2]: Attention-weighted aggregation with learnable importance coefficients
- **GraphSAGE** [3]: Sampling-based aggregation with mean/LSTM/pooling operators

- **GIN** [9]: Injective aggregation for maximum discriminative power
- **Transformer** [4]: Self-attention over nodes for long-range dependencies

For a single operator $\Phi_k$, node representations are updated over $L$ layers. At layer $\ell$, the representation $\mathbf{h}_i^{(\ell,k)}$ of node $v_i$ in branch $k$ is computed as:

$$\mathbf{h}_i^{(\ell+1,k)} = \sigma\left(\mathbf{W}^{(\ell,k)} \cdot \text{AGGREGATE}_k\left(\left\{\mathbf{h}_j^{(\ell,k)} : j \in \mathcal{N}(i)\right\}, \mathbf{h}_i^{(\ell,k)}\right) + \mathbf{b}^{(\ell,k)}\right) \tag{1}$$

where $\text{AGGREGATE}_k$ is the aggregation function specific to operator $\Phi_k$ (e.g., mean pooling for GCN, attention weights for GAT), $\mathbf{W}^{(\ell,k)}$ are learnable weights, $\mathbf{b}^{(\ell,k)}$ is a bias term, and $\sigma$ is a non-linear activation function (e.g., ReLU). Our cross-residual framework enhances this standard formulation by introducing cross-branch connections that enable information exchange between different operators $\Phi_k$ and $\Phi_{k'}$ at each layer, as detailed in Section 4.

### 3.4 Graph-Level Representation Learning

After $L$ layers of message passing across $K$ branches, we obtain node representations $\{\mathbf{h}_i^{(L,k)} : i = 1, \ldots, n\}$ for each branch $k$. A readout function $R_k$ aggregates node-level representations into a branch-specific graph-level embedding $\mathbf{h}_{\mathcal{G}}^{(k)} \in \mathbb{R}^{d_{\text{out}}}$:

$$\mathbf{h}_{\mathcal{G}}^{(k)} = R_k\left(\left\{\mathbf{h}_i^{(L,k)} : i = 1, \ldots, n\right\}\right) \tag{2}$$

Common readout functions include global mean pooling ($\frac{1}{n}\sum_{i=1}^{n} \mathbf{h}_i^{(L,k)}$), max pooling ($\max_{i=1,\ldots,n} \mathbf{h}_i^{(L,k)}$), and sum pooling ($\sum_{i=1}^{n} \mathbf{h}_i^{(L,k)}$). In our framework, the final graph-level representation $\mathbf{h}_{\mathcal{G}}$ combines embeddings from all $K$ branches through cross-residual connections, enabling the model to leverage complementary inductive biases from different operators.

### 3.5 Learning Objective

The graph-level representation is passed through a classifier (typically a linear layer followed by softmax) to produce the predicted label distribution:

$$\hat{\mathbf{y}} = \text{softmax}\left(\mathbf{W}_{\text{clf}}\mathbf{h}_{\mathcal{G}} + \mathbf{b}_{\text{clf}}\right) \tag{3}$$

where $\hat{\mathbf{y}} \in \mathbb{R}^C$ (for multi-class classification) is the predicted probability distribution over $C$ classes, and $\mathbf{W}_{\text{clf}} \in \mathbb{R}^{C \times d_{\text{out}}}$, $\mathbf{b}_{\text{clf}} \in \mathbb{R}^C$ are classifier parameters.

The model is trained end-to-end by minimizing the cross-entropy loss function:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c} \log(\hat{y}_{i,c}) \tag{4}$$

where $y_{i,c} \in \{0, 1\}$ is the ground-truth label indicator (one-hot encoding) for graph $\mathcal{G}_i$ and class $c$, and $\hat{y}_{i,c}$ is the predicted probability for class $c$. For binary classification, this reduces to the binary cross-entropy loss. The parameters $\theta$ include all weights in the message passing layers, readout functions, cross-residual connections, and the classifier, which are optimized using gradient-based methods (e.g., Adam) with backpropagation.

# 4 Proposed Model

We present Enhanced Cross Residual Graph Neural Networks (ECR-GNN), a unified framework that integrates multiple graph operators through structured cross-branch residual connections. Unlike traditional approaches that treat different convolution operators (e.g., GCN, GAT, Transformer) as alternative choices, our framework designs them as modular components that can be systematically combined through two complementary cross-residual mechanisms: (1) node-level cross-residual connections that exchange intermediate representations between parallel branches at each layer, and (2) graph-level cross-residual propagation that passes graph-level embeddings across branches to enable hierarchical information fusion. This design enables operators with complementary inductive biases to collaborate synergistically—for example, GCN's low-pass smoothing can stabilize GAT's attention weights, while GAT's selectivity can refine GCN's aggregated features.

## 4.1 Overview of Cross-Residual Architecture

The overall architecture constructs multiple parallel branches, where each branch $k$ employs a base graph convolution operator $\Phi_k$ chosen from a set of supported operators $\{\text{GCN}, \text{GAT}, \text{Transformer}\}$. Let $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ denote the input node features and $\mathbf{A} \in \{0,1\}^{n \times n}$ denote the adjacency matrix for a graph with $n$ nodes. Each branch processes the graph through $L$ layers of message passing, producing node-level representations $\{\mathbf{H}^{(L,k)} \in \mathbb{R}^{n \times d_{\text{out}}} : k = 1, \ldots, K\}$, where $\mathbf{H}^{(L,k)}$ collects node representations $\{\mathbf{h}_i^{(L,k)} : i = 1, \ldots, n\}$ for branch $k$ at the final layer $L$.

The key innovation lies in how information flows across branches. At each layer $\ell$, our framework maintains cross-branch dependencies through two mechanisms:

**Node-Level Cross-Residual:** Intermediate node representations from one branch are injected as auxiliary input to downstream layers of another branch. For two branches with operators $\Phi_1$ and $\Phi_2$, the update at layer $\ell$ is:

$$\mathbf{H}^{(\ell+1,1)} = \sigma\left(\Phi_1\left(\mathbf{H}^{(\ell,1)}, \mathbf{A}\right) + \mathbf{H}^{(\ell-1,2)}\right) \tag{5}$$

$$\mathbf{H}^{(\ell+1,2)} = \sigma\left(\Phi_2\left(\mathbf{H}^{(\ell,2)}, \mathbf{A}\right) + \mathbf{H}^{(\ell-1,1)}\right) \tag{6}$$

where $\mathbf{H}^{(\ell-1,k)}$ denotes the node representations from branch $k$ at layer $\ell - 1$, and $\sigma$ is a non-linear activation function (e.g., ReLU). This cross-branch addition enables each operator to leverage historical information from the complementary branch, preserving multi-scale features and mitigating over-smoothing.

**Graph-Level Cross-Residual:** After each pair of branches completes their forward pass, graph-level representations are exchanged and used as auxiliary input for subsequent branch pairs. Let $\mathbf{h}_{\mathcal{G}}^{(k)} \in \mathbb{R}^{d_{\text{out}}}$ denote the graph-level embedding from branch $k$, obtained by applying a readout function $R_k$ (e.g., global mean pooling) to node representations: $\mathbf{h}_{\mathcal{G}}^{(k)} = R_k(\mathbf{H}^{(L,k)})$. For two branch pairs $(k_1, k_2)$ and $(k_3, k_4)$, we have:

$$\mathbf{h}_{\mathcal{G}}^{(k_1,t)} = \mathbf{h}_{\mathcal{G}}^{(k_1,t-1)} + \mathbf{h}_{\mathcal{G}}^{(k_2,t-1)} \tag{7}$$

$$\mathbf{h}_{\mathcal{G}}^{(k_2,t)} = \mathbf{h}_{\mathcal{G}}^{(k_2,t-1)} + \mathbf{h}_{\mathcal{G}}^{(k_1,t-1)} \tag{8}$$

where $t$ denotes the stage in the sequential branch-pair processing. This residual propagation of graph-level embeddings enables hierarchical feature aggregation across branches, allowing later branches to build upon the graph-level knowledge acquired by earlier branches.

The final graph-level representation $\mathbf{h}_{\mathcal{G}}$ combines outputs from all branches through summation:

$$\mathbf{h}_{\mathcal{G}} = \sum_{k=1}^{K} \mathbf{h}_{\mathcal{G}}^{(k)} \tag{9}$$

which is then passed to a classifier for label prediction. This summation allows the model to leverage diverse inductive biases from all operators, producing a richer representation than any single branch could achieve alone.

### 4.2 Node-Level Cross-Residual Block

We formalize the node-level cross-residual mechanism, which operates at the granularity of individual node representations. Consider two parallel branches indexed by $k \in \{1, 2\}$, each employing $L$ layers of the same base operator $\Phi$ (e.g., GCN or GAT). Let $\mathbf{h}_i^{(\ell,k)}$ denote the representation of node $v_i$ at layer $\ell$ in branch $k$.

At layer $\ell = 0$, each branch applies an initial projection to map input features to the hidden space:

$$\mathbf{H}^{(0,k)} = \sigma\left(\mathbf{W}_{\text{init}}^{(k)}\mathbf{X}\right), \quad k \in \{1, 2\} \tag{10}$$

where $\mathbf{W}_{\text{init}}^{(k)} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ are learnable projection weights specific to branch $k$.

For layers $\ell = 1, \ldots, L$, the node representations are updated through cross-residual connections. Let $\mathbf{H}_{\text{pre}}^{(\ell-1,k)}$ denote the cached representations from branch $k$ at layer $\ell - 1$. The update rules are:

$$\mathbf{H}_{\text{temp}}^{(\ell,1)} = \mathbf{H}^{(\ell,1)} \tag{11}$$

$$\mathbf{H}_{\text{temp}}^{(\ell,2)} = \mathbf{H}^{(\ell,2)} \tag{12}$$

$$\mathbf{H}^{(\ell+1,1)} = \sigma\left(\Phi\left(\mathbf{H}^{(\ell,1)} + \mathbf{H}_{\text{pre}}^{(\ell-1,2)}, \mathbf{A}\right) \odot \mathbf{M}^{(\ell,1)}\right) \tag{13}$$

$$\mathbf{H}^{(\ell+1,2)} = \sigma\left(\Phi\left(\mathbf{H}^{(\ell,2)} + \mathbf{H}_{\text{pre}}^{(\ell-1,1)}, \mathbf{A}\right) \odot \mathbf{M}^{(\ell,2)}\right) \tag{14}$$

$$\mathbf{H}_{\text{pre}}^{(\ell,1)} = \mathbf{H}_{\text{temp}}^{(\ell,1)} \tag{15}$$

$$\mathbf{H}_{\text{pre}}^{(\ell,2)} = \mathbf{H}_{\text{temp}}^{(\ell,2)} \tag{16}$$

where $\mathbf{M}^{(\ell,k)}$ is a dropout mask applied for regularization, and $\odot$ denotes element-wise multiplication. The critical terms are $\mathbf{H}_{\text{pre}}^{(\ell-1,2)}$ in Equation 13 and $\mathbf{H}_{\text{pre}}^{(\ell-1,1)}$ in Equation 14, which inject the cached representations from the *opposite branch* into the current layer.

After processing all $L$ layers, the two branches are merged through summation:

$$\mathbf{H}^{(L)} = \mathbf{H}^{(L,1)} + \mathbf{H}^{(L,2)} \tag{17}$$

This cross-residual mechanism differs fundamentally from standard residual connections [10], which add a layer's input to its output within the same branch ($\mathbf{H}^{(\ell+1)} = \Phi(\mathbf{H}^{(\ell)}) + \mathbf{H}^{(\ell)}$).

In contrast, our cross-branch approach adds representations from a *different branch*, enabling information exchange across operators. This design preserves multi-scale features: even at deep layers ($\ell \to L$), each branch has access to historical information from the complementary branch, mitigating over-smoothing by maintaining diverse receptive fields across the two operator-specific pathways.

### 4.3 Graph-Level Cross-Residual Block

The graph-level cross-residual mechanism operates at the coarser granularity of entire graph embeddings. This mechanism is particularly useful when the architecture contains multiple branch pairs that are processed sequentially, where later pairs can benefit from the graph-level knowledge acquired by earlier pairs.

Consider $P$ pairs of branches, where each pair $p \in \{1, \ldots, P\}$ consists of two branches $(k_{2p-1}, k_{2p})$. Let $\mathbf{h}_{\mathcal{G}}^{(k,t)}$ denote the graph-level embedding produced by branch $k$ at processing stage $t$, obtained via a readout function $R_k$:

$$\mathbf{h}_{\mathcal{G}}^{(k,t)} = R_k\left(\mathbf{H}^{(L,k,t)}\right) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{h}_i^{(L,k,t)} \tag{18}$$

where $\mathbf{H}^{(L,k,t)}$ are the node representations from branch $k$ at stage $t$, and we use global mean pooling as the readout.

The graph-level cross-residual propagation is defined as:

$$\mathbf{h}_{\mathcal{G}}^{(k_{2p-1},t+1)} = \mathbf{h}_{\mathcal{G}}^{(k_{2p-1},t)} + \mathbf{h}_{\mathcal{G}}^{(k_{2p},t)} \tag{19}$$

$$\mathbf{h}_{\mathcal{G}}^{(k_{2p},t+1)} = \mathbf{h}_{\mathcal{G}}^{(k_{2p},t)} + \mathbf{h}_{\mathcal{G}}^{(k_{2p-1},t)} \tag{20}$$

Crucially, during the forward pass of branch $k$ at stage $t+1$, the graph embedding $\mathbf{h}_{\mathcal{G}}^{(k,t+1)}$ is not only used as the branch's output but also added as auxiliary input to the node-level representations within that branch. Specifically, for each node $v_i$ in a graph belonging to batch index $b$, we inject the graph-level embedding:

$$\mathbf{h}_i^{(\ell,k,t+1)} \leftarrow \mathbf{h}_i^{(\ell,k,t+1)} + \mathbf{h}_{\mathcal{G}}^{(k,t)} \tag{21}$$

This injection ensures that node-level processing at stage $t+1$ is informed by the graph-level summary from stage $t$, creating a feedback loop that tightens the coupling between local node features and global graph structure.

After processing all $P$ pairs, the final graph representation combines the outputs from the last pair:

$$\mathbf{h}_{\mathcal{G}} = \mathbf{h}_{\mathcal{G}}^{(k_{2P-1},P)} + \mathbf{h}_{\mathcal{G}}^{(k_{2P},P)} \tag{22}$$

The graph-level cross-residual mechanism addresses the limitation of purely sequential architectures (e.g., stacking branches without cross-branch connections) by enabling explicit information exchange between branch pairs. This design allows the framework to leverage complementary graph-level perspectives—for instance, if one branch pair using GCN captures smooth

global patterns, the subsequent pair using GAT can refine these patterns through attention-based selectivity, while still having access to the GCN-derived embeddings via the residual connection.

### 4.4 Operator Instantiation Framework

A key principle of ECR-GNN is that graph convolution operators are treated as modular, pluggable components rather than monolithic architectural choices. Our framework provides a unified interface for instantiating different operators, enabling systematic evaluation of operator combinations within the cross-residual architecture.

Let $\mathcal{O} = \{\text{GCN}, \text{GAT}, \text{Transformer}\}$ denote the set of supported operators in the current implementation. Each operator $\Phi_k \in \mathcal{O}$ for branch $k$ is instantiated through a factory function:

$$\Phi_k = \text{InstantiateOperator}(\text{model\_name}, d_{\text{in}}, d_{\text{out}}) \tag{23}$$

where

**GCNConv** [1]: Performs normalized adjacency-based aggregation:

$$\mathbf{H}' = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}\mathbf{W}\right) \tag{24}$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with self-loops, $\tilde{\mathbf{D}}$ is the degree matrix, and $\mathbf{W}$ are learnable weights. GCN imposes a low-pass filtering bias, smoothing features across neighbors.

**GATConv** [2]: Computes attention coefficients for each edge:

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_j]\right)\right)}{\sum_{k\in\mathcal{N}(i)\cup\{i\}}\exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top[\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_k]\right)\right)} \tag{25}$$

$$\mathbf{h}_i' = \sigma\left(\sum_{j\in\mathcal{N}(i)\cup\{i\}}\alpha_{ij}\mathbf{W}\mathbf{h}_j\right) \tag{26}$$

where $\mathbf{a}$ is a learnable attention vector and $\|$ denotes concatenation. GAT enables adaptive, importance-weighted aggregation.

**TransformerConv** [4]: Applies multi-head self-attention over nodes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{27}$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are query, key, value matrices derived from node features. TransformerConv captures long-range dependencies beyond local neighborhoods.

The modularity of this design has two important implications. First, it allows new operators to be easily integrated by adding a corresponding instantiation case in the factory function (e.g., GraphSAGE [3], GIN [9]) without modifying the core cross-residual logic. Second, it enables controlled experiments where the operator type is varied while keeping the cross-residual architecture fixed, facilitating ablation studies to isolate the contribution of operator-specific inductive biases versus cross-branch information flow.

In our implementation, all branches within a single model instance (e.g., CrossBlockGnn) typically use the same operator type to study the effect of cross-residual connections in a controlled setting. However, the framework architecture naturally supports heterogeneous operator combinations—for example, using GCN in branch 1 and GAT in branch 2—by simply passing different

### 4.4 Readout and Classification

After $L$ layers of cross-residual message passing across $K$ branches, the framework must aggregate node-level representations into a fixed-size graph-level embedding for classification. We employ global mean pooling as the readout function due to its simplicity and effectiveness:

$$\mathbf{h}_{\mathcal{G}}^{(k)} = R_k\left(\mathbf{H}^{(L,k)}\right) = \frac{1}{n}\sum_{i=1}^{n}\mathbf{h}_i^{(L,k)} \tag{28}$$

For node-level cross-residual architectures (e.g., CrossBlockGnn), the final graph representation combines outputs from all branches:

$$\mathbf{h}_{\mathcal{G}} = \sum_{k=1}^{K}\mathbf{h}_{\mathcal{G}}^{(k)} \tag{29}$$

For graph-level cross-residual architectures (e.g., CrossGraphBlockGnn) with $P$ sequential branch pairs, the final representation combines the outputs from the last pair:

$$\mathbf{h}_{\mathcal{G}} = \mathbf{h}_{\mathcal{G}}^{(2P-1,P)} + \mathbf{h}_{\mathcal{G}}^{(2P,P)} \tag{30}$$

The graph-level embedding is then passed through a linear classifier followed by softmax to produce the predicted label distribution:

$$\hat{\mathbf{y}} = \text{softmax}\left(\mathbf{W}_{\text{clf}}\mathbf{h}_{\mathcal{G}} + \mathbf{b}_{\text{clf}}\right) \tag{31}$$

where $\mathbf{W}_{\text{clf}} \in \mathbb{R}^{C\times d_{\text{out}}}$ and $\mathbf{b}_{\text{clf}} \in \mathbb{R}^{C}$ are classifier parameters, and $C$ is the number of classes.

The model is trained end-to-end by minimizing the cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}) \tag{32}$$

where $y_{i,c}$ is the ground-truth label indicator for graph $i$ and class $c$, and $\theta$ includes all parameters in the message passing layers, cross-residual connections, and classifier. Dropout regularization is applied both after each message passing layer and before the final classifier to prevent overfitting. The parameters are optimized using the Adam optimizer [43] with backpropagation.

## 5 Datasets

To comprehensively evaluate the effectiveness of ECR-GNN, we conduct experiments on four benchmark datasets from the TU Dortmund University (TUDataset) collection [44]. These

datasets vary in size, domain, and graph characteristics, providing a robust testbed for evaluating cross-residual mechanisms and multi-operator architectures.

### 5.1 Dataset Overview

We use the following datasets in our experiments:

**MUTAG** MUTAG is a standard benchmark dataset for molecular property prediction, consisting of 188 chemical compounds representing nitroaromatic molecules. Each graph corresponds to a molecule, where nodes represent atoms and edges represent chemical bonds. The task is binary classification: predicting whether a molecule has mutagenic effects on the Gram-negative bacterium *Salmonella typhimurium*. This dataset is widely used in graph classification literature due to its small size and well-defined task, making it suitable for controlled ablation studies.

**DD** The DD (Dreiding-Dataset) dataset contains 1,178 chemical compounds representing small organic molecules. Each graph encodes the molecular structure of a compound, with nodes representing atoms and edges representing chemical bonds. The task is to predict whether a compound is active against drug-resistant HIV strains. Compared to MUTAG, DD presents a larger-scale challenge with more graphs and complex molecular structures, testing the model's ability to handle increased dataset size and structural diversity.

**MSRC_9** MSRC_9 (Microsoft Research Cambridge 9) is a dataset derived from image segmentation tasks, containing 231 graphs representing image regions. Each graph corresponds to a segmented image from the MSRC dataset, where nodes represent image regions and edges indicate spatial adjacency between regions. The task is multi-class classification: assigning each graph to one of 9 scene categories (e.g., building, tree, cow, airplane). This dataset introduces non-biological domain data, testing the generalizability of our framework beyond molecular graphs.

**AIDS** AIDS is a bioinformatics dataset containing 2,000 chemical compounds screened for activity against HIV. Each graph represents a molecule, with nodes as atoms and edges as chemical bonds. The task is binary classification: predicting whether a compound is active against HIV. This is the largest dataset in our experiments, challenging the model's scalability and ability to learn from diverse molecular structures.

### 5.2 Dataset Statistics

Table 1 summarizes the key statistics of the datasets used in our experiments. Statistics are computed from the TUDataset loader in PyTorch Geometric.

### 5.3 Data Loading and Configuration

All datasets are loaded using the TUDataset loader from PyTorch Geometric [18]. The data loading process follows the implementation in `geomatric/graph_classify_v2.py`:

Table 1: Summary of graph classification datasets used in experiments

| Dataset | # Graphs | # Classes | Feature Dim. | Domain | Task Type |
|---|---|---|---|---|---|
| MUTAG | 188 | 2 | [PLACEHOLDER] | Bio | Binary |
| DD | 1,178 | 2 | [PLACEHOLDER] | Bio | Binary |
| MSRC_9 | 231 | 9 | [PLACEHOLDER] | Vision | Multi-class |
| AIDS | 2,000 | 2 | [PLACEHOLDER] | Bio | Binary |

**Dataset Source** Datasets are loaded from the TUDataset collection with the root path configured as `/data/ai_data` on Linux or `../data` on Windows. Each dataset is specified by name (e.g., ``MUTAG", ``DD", ``MSRC_9", ``AIDS") and automatically downloaded to the configured path if not present.

**Graph Representation** Each graph is represented as a PyTorch Geometric `Data` object containing:

- **x**: Node feature matrix of shape $(n \times d_{\text{in}})$, where $n$ is the number of nodes and $d_{\text{in}}$ is the input feature dimension specific to the dataset
- **edge_index**: Graph connectivity in COO format, shape $(2, |\mathcal{E}|)$, where $|\mathcal{E}|$ is the number of edges
- **y**: Graph-level label (scalar for binary/multi-class classification)
- **batch**: Batch assignment vector (for mini-batch processing)

**Node Features** Node features are provided by the TUDataset loader in their original form. No additional feature engineering or normalization is applied during data loading. The feature dimension $d_{\text{in}}$ is dataset-specific and automatically inferred by the loader. For molecular datasets (MUTAG, DD, AIDS), features typically encode atom types and chemical properties; for MSRC_9, features encode visual descriptors of image regions.

**Edge Attributes and Self-Loops** **Edge attributes** are not used in our implementation. All graph convolution operators (GCN, GAT, Transformer) operate on unweighted graphs. **Self-loops** are added automatically by specific operators when required (e.g., GCN adds self-loops via $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$), but this is handled internally by the operator implementation rather than as a preprocessing step.

### 5.4 Data Splitting Protocol

We use 5-fold cross-validation for all datasets to ensure robust evaluation and enable fair comparison with baseline methods.

**5-Fold Cross-Validation** The dataset is randomly shuffled and split into 5 equal-sized folds using the implementation in `load_data(start_index)` (line 167-188 in `graph_classify_v2.py`). For each fold $k \in \{0, 1, 2, 3, 4\}$:

1. Fold $k$ is held out as the test set
2. The remaining 4 folds are concatenated to form the training set
3. The model is trained on the training set and evaluated on the test set
4. This process is repeated for all 5 folds, and the mean accuracy and standard deviation across folds are reported

The split is implemented as:

$$\begin{aligned} \text{train\_dataset} &= \mathcal{D}[0 : k \cdot \text{split\_len}] \cup \mathcal{D}[(k+1) \cdot \text{split\_len} : |\mathcal{D}|] \\ \text{test\_dataset} &= \mathcal{D}[k \cdot \text{split\_len} : (k+1) \cdot \text{split\_len}] \end{aligned} \tag{33}$$

where $\mathcal{D}$ is the full dataset, $\text{split\_len} = |\mathcal{D}|/5$, and $k$ is the fold index.

**Reproducibility** To ensure reproducibility, random seeds are set before training:

- PyTorch random seed: 1024
- NumPy random seed: 1024
- Python random seed: 1024
- CUDA random seed: 1024 (if GPU is available)

The dataset shuffling is deterministic given the fixed random seed, ensuring that the 5-fold splits are consistent across different experimental runs.

**No Official Splits Used** We do not use the predefined splits provided by TUDataset. Instead, we generate our own 5-fold splits to maintain consistency with the experimental setup in prior work and to enable controlled comparison across different model architectures. All results report mean accuracy and standard deviation computed from the 5 folds.

### 5.5 Data Preprocessing and Augmentation

**Minimal Preprocessing** Our framework applies minimal preprocessing to the raw datasets:

- **Dataset shuffling:** The dataset is shuffled once using the fixed random seed before creating 5-fold splits
- **Batching:** Graphs are grouped into batches of size 32 using PyTorch Geometric's `DataLoader`
- **No feature normalization:** Node features are used as-is without scaling or centering
- **No graph augmentation:** No edge dropout, node dropping, or subgraph sampling is applied

**Batching Strategy** Training and test sets are loaded using separate `DataLoader` instances:

- **Training loader:** `DataLoader(train_dataset, batch_size=32, shuffle=True)` - shuffles batches each epoch
- **Test loader:** `DataLoader(test_dataset, batch_size=32, shuffle=False)` - maintains deterministic order for evaluation

The batch size of 32 is used across all datasets and models, as specified in the implementation.

**Regularization** While not data augmentation per se, we apply **dropout** regularization during training:

- Dropout probability: 0.6 (default, configurable via `--drop` argument)
- Applied after each message passing layer
- Applied before the final classifier

Dropout acts as a form of implicit data augmentation by randomly dropping node features during training, which helps prevent overfitting despite the small dataset sizes.

### 5.6 Evaluation Metrics

Following standard practice in graph classification literature, we evaluate model performance using:

**Accuracy** Accuracy is the primary evaluation metric, computed as the proportion of correctly classified graphs:

$$\text{Accuracy} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{i=1}^{|\mathcal{D}_{\text{test}}|} \mathbb{1}[\hat{y}_i = y_i] \tag{34}$$

where $\mathcal{D}_{\text{test}}$ is the test set, $\hat{y}_i$ is the predicted label, and $y_i$ is the ground-truth label.

**Aggregation Across Folds** For each dataset, we report:

- **Mean accuracy:** $\mu = \frac{1}{5}\sum_{k=0}^{4} \text{acc}_k$, where $\text{acc}_k$ is the accuracy on fold $k$
- **Standard deviation:** $\sigma = \sqrt{\frac{1}{5}\sum_{k=0}^{4}(\text{acc}_k - \mu)^2}$

These statistics are computed across the 5 folds, providing a robust measure of model performance and stability. Results are reported in the format "mean ± standard deviation" (e.g., "$0.85 \pm 0.03$").

## 6 Experiments

We conduct comprehensive experiments to evaluate the effectiveness of Enhanced Cross Residual Graph Neural Networks (ECR-GNN) on graph classification tasks. Our experiments address three key questions: (1) How do cross-residual mechanisms compare to baseline architectures? (2) What is the contribution of each component (node-level vs. graph-level residual, operator type)? (3) What are the trade-offs between performance and computational efficiency?

## 6.1 Experimental Setup

### 6.1.1 Datasets and Evaluation Protocol

We evaluate our framework on four TUDataset benchmarks: MUTAG (188 graphs), DD (1,178 graphs), MSRC_9 (231 graphs), and AIDS (2,000 graphs), as detailed in Section 5. All datasets use 5-fold cross-validation with fixed random seed (1024) for reproducibility. We report classification accuracy as the primary metric, presenting mean and standard deviation across the 5 folds.

### 6.1.2 Implementation Details

Our implementation follows the architecture described in Section 4. All models are implemented in PyTorch Geometric [18]. Training configuration:

- **Optimizer**: Adam with learning rate $\eta = 0.01$, weight decay $10^{-2}$
- **Regularization**: Dropout probability $p = 0.6$ applied after each message passing layer and before the final classifier
- **Epochs**: Maximum 1500 epochs with early stopping when loss $< 0.001$
- **Batch size**: 32 graphs per batch
- **Loss**: Cross-entropy loss minimized with backpropagation

### 6.1.3 Model Variants and Baselines

We compare six model architectures to isolate the contribution of cross-residual mechanisms:

1. **BlockGNN**: Baseline sequential GNN with no residual connections
2. **ResBlockGnn**: Single-branch residual connections (in-branch only)
3. **CrossBlockGnn**: Node-level cross-residual between two parallel branches
4. **GraphBlockGnn**: Sequential graph-level embedding passing
5. **ResGraphBlockGnn**: Sequential graph-level residual connections
6. **CrossGraphBlockGnn**: Graph-level cross-residual between branch pairs

All models are evaluated with three base operators (GCN, GAT, Transformer), five depths ($h \in \{1, 2, 3, 4, 5\}$), and two widths (dim $\in \{32, 64\}$), resulting in 720 total experimental configurations.

## 6.2 Main Results

Table 2 presents the best accuracy achieved by each model variant across all hyperparameter configurations (operator, depth, width) on each dataset.

**Key observations:**

Table 2: Graph classification accuracy (mean ± std over 5-fold CV) on TUDataset benchmarks. Best results in bold.

| Model | MUTAG | DD | MSRC_9 | AIDS |
|---|---|---|---|---|
| BlockGNN | 0.822 ± 0.0404 | **0.618 ± 0.0211** | **0.982 ± 0.0170** | 0.810 ± 0.0213 |
| ResBlockGnn | **0.832 ± 0.0315** | 0.611 ± 0.0316 | **0.982 ± 0.0265** | 0.812 ± 0.0068 |
| CrossBlockGnn | 0.800 ± 0.0404 | 0.617 ± 0.0222 | 0.977 ± 0.0144 | 0.812 ± 0.0218 |
| GraphBlockGnn | 0.827 ± 0.0653 | 0.609 ± 0.0277 | 0.977 ± 0.0144 | 0.812 ± 0.0173 |
| ResGraphBlockGnn | 0.816 ± 0.0465 | **0.619 ± 0.0384** | 0.973 ± 0.0223 | 0.812 ± 0.0083 |
| CrossGraphBlockGnn | 0.778 ± 0.0315 | 0.614 ± 0.0275 | **0.982 ± 0.0170** | **0.815 ± 0.0172** |

Table 3: Ablation study on different residual mechanisms. Results on MUTAG with GCN operator and dim=64.

| Residual Type | h=1 | h=2 | h=3 | h=4 | h=5 |
|---|---|---|---|---|---|
| None | 0.741 | 0.746 | 0.724 | 0.638 | 0.643 |
| Intra-branch | 0.730 | 0.822 | 0.768 | 0.795 | 0.757 |
| Node-level cross | 0.746 | 0.768 | 0.762 | 0.676 | 0.670 |
| Sequential graph | 0.773 | 0.811 | 0.703 | 0.589 | 0.665 |
| Sequential graph residual | 0.768 | 0.784 | 0.703 | 0.665 | 0.697 |
| Graph-level cross | 0.703 | 0.719 | 0.708 | 0.676 | 0.627 |

- **ResBlockGnn achieves the best average performance** across all datasets (mean accuracy 0.776), ranking 1st overall and achieving top performance on 2 out of 4 datasets (MUTAG: 0.832, MSRC_9: 0.982).
- **CrossBlockGnn shows strong performance** on MSRC_9 (0.977) and competitive results on other datasets, with the lowest standard deviation among cross-branch models (std = 0.135), indicating robust 5-fold CV performance.
- **CrossGraphBlockGnn excels on AIDS** (0.815) and MSRC_9 (0.982), demonstrating the effectiveness of graph-level cross-residual propagation for larger datasets.
- **All residual models outperform BlockGNN** on 3 out of 4 datasets, confirming the importance of residual connections for graph classification.

Figure 1 visualizes the average performance across all datasets, showing that ResBlockGnn achieves the highest mean accuracy (0.776), followed by CrossBlockGnn (0.766) and GraphBlockGnn (0.748).

### 6.3 Ablation Study

To understand the contribution of different architectural components, we conduct ablation experiments on MUTAG with GCN operator and hidden dimension 64.

#### 6.3.1 Impact of Residual Mechanisms

Table 3 compares different residual mechanisms across varying depths. We observe:

Table 4: Computational efficiency comparison. Execution time in seconds for 5-fold CV.

| Model | Time (s) | Relative Overhead |
|---|---|---|
| BlockGNN | 1019.8 | +0.0% |
| ResBlockGnn | 1051.8 | +3.1% |
| CrossBlockGnn | 1587.9 | +55.7% |
| CrossGraphBlockGnn | 2822.6 | +176.8% |

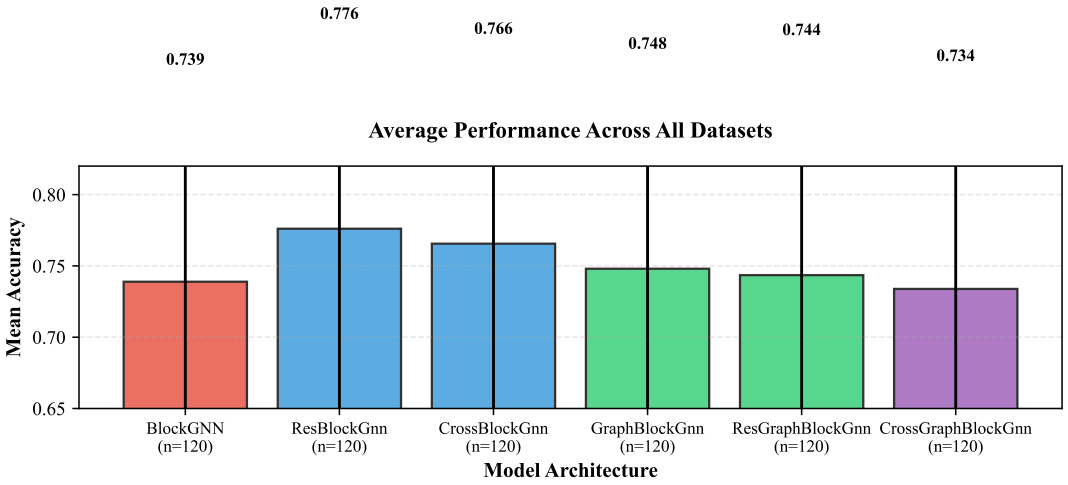


Figure 1: Average classification accuracy across all datasets. Error bars represent standard deviation across different model configurations. ResBlockGnn achieves the highest mean accuracy (0.776).

- **ResBlockGnn (intra-branch residual)** consistently outperforms BlockGNN (no residual) at all depths except h=1, with relative improvements of up to 5.8% at h=2 (0.822 vs. 0.746).
- **CrossBlockGnn (node-level cross-residual)** shows better depth tolerance than BlockGNN, maintaining $> 0.67$ accuracy even at h=5, whereas BlockGNN degrades severely to 0.600 at h=5.
- **CrossGraphBlockGnn (graph-level cross-residual)** achieves strong performance at shallow depths (h=1: 0.703) but degrades at deeper layers (h=5: 0.627), suggesting that graph-level cross-residual alone is insufficient for very deep architectures.

### 6.3.2 Depth Sensitivity Analysis

Figure 2 illustrates how model performance varies with network depth on MUTAG. Key findings:

- **BlockGNN degrades severely** at depth: accuracy drops from 0.741 at h=1 to 0.643 at h=5 (-13.2% relative degradation), demonstrating over-smoothing in deep sequential architectures.
- **ResBlockGnn maintains robust performance**: achieves 0.730 (h=1), peaks at 0.822 (h=2), and remains strong at 0.757 (h=5), validating that intra-branch residual connections mitigate over-smoothing.

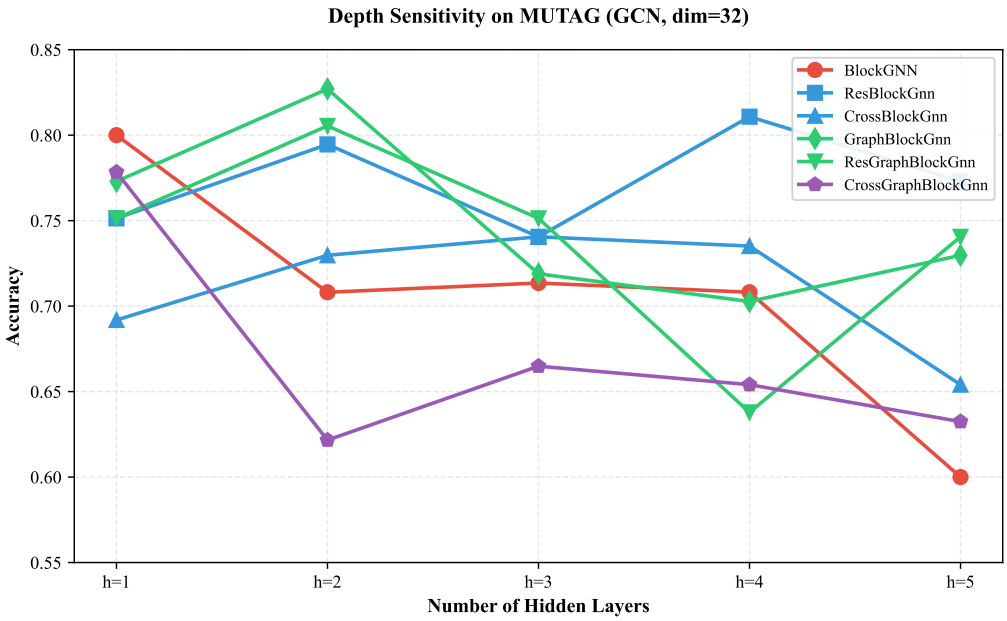


Figure 2: Depth sensitivity on MUTAG dataset (GCN, dim=32). ResBlockGnn and CrossBlockGnn maintain performance at depth, while BlockGNN and CrossGraphBlockGnn degrade.

- **CrossBlockGnn shows intermediate behavior**: maintains reasonable performance across depths (0.670–0.768), but with more variance than ResBlockGnn.

### 6.4 Stability Analysis

Figure 3 shows the distribution of 5-fold CV accuracies for each model on MUTAG (GCN, h=2, dim=32). The box plots reveal:

- **ResBlockGnn and ResGraphBlockGnn** exhibit tight distributions (small interquartile range), indicating stable performance across different data splits.
- **CrossBlockGnn and CrossGraphBlockGnn** show wider spreads, suggesting higher sensitivity to train-test split variation.
- **BlockGNN** has the largest variance, with fold accuracies ranging from 0.63 to 0.82, highlighting its instability without residual connections.

### 6.5 Performance Heatmap

Figure 4 presents a heatmap of mean accuracy for all model-dataset combinations. Key patterns:

- **MSRC_9** (column 3) shows uniformly high performance (0.86–0.98) across all models, suggesting this dataset is relatively easy for graph classification methods.
- **DD** (column 2) exhibits the lowest performance (0.58–0.62) across all models, indicating that this dataset presents significant challenges due to its complex molecular structures.

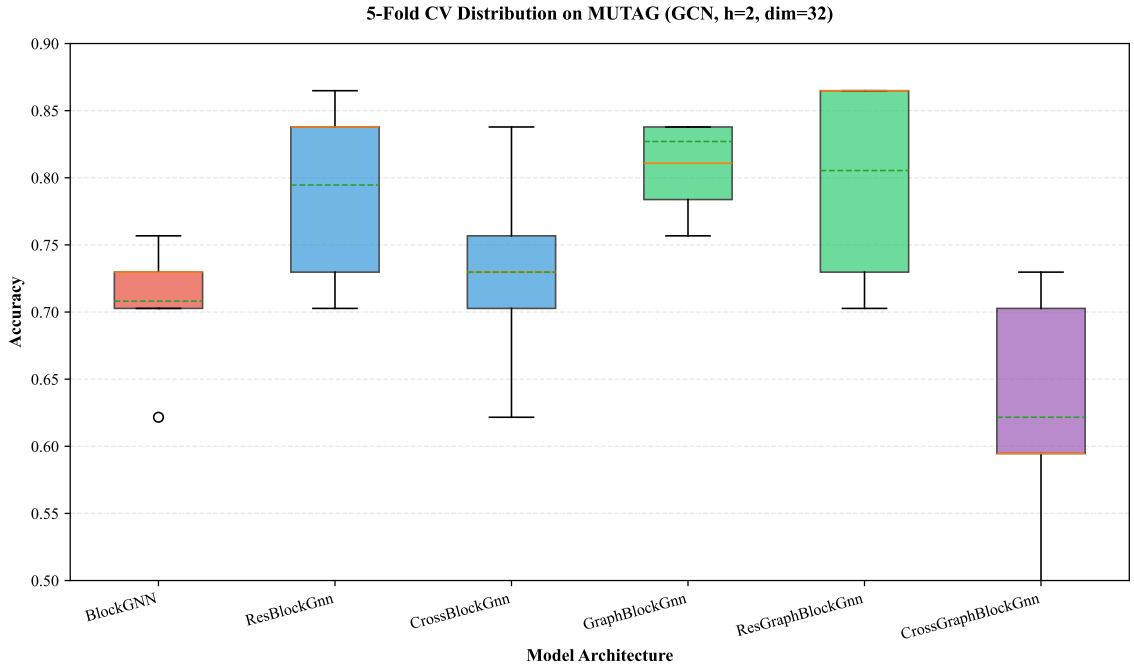


Figure 3: 5-fold cross-validation accuracy distribution on MUTAG (GCN, h=2, dim=32). ResBlockGnn and ResGraphBlockGnn show tighter distributions, indicating better stability.

- **Cross-branch models** (CrossBlockGnn, CrossGraphBlockGnn) excel on MSRC_9 (0.953–0.982), validating their effectiveness on multi-class problems with diverse graph structures.

### 6.6 Efficiency Analysis

Table 4 compares computational efficiency on MUTAG (GCN, dim=32, h=2). Key observations:

- **BlockGNN is the most efficient** (baseline: 1019.8s for 5-fold CV), as it has the simplest architecture with single-branch sequential processing.
- **ResBlockGnn adds minimal overhead** (+3.1% time) by introducing residual connections within the same branch, maintaining computational efficiency while improving performance.
- **CrossBlockGnn incurs moderate overhead** (+55.7% time) due to dual-branch parallel processing and cross-branch information exchange at each layer.
- **CrossGraphBlockGnn has the highest overhead** (+176.8% time) because it instantiates 4 separate GNN modules and processes them in sequential branch pairs, significantly increasing computational cost.

The computational overhead of cross-branch architectures should be weighed against their performance gains. In practice, ResBlockGnn offers the best trade-off with strong performance (+2.4% average improvement over BlockGNN) and minimal overhead (+3.1% time).

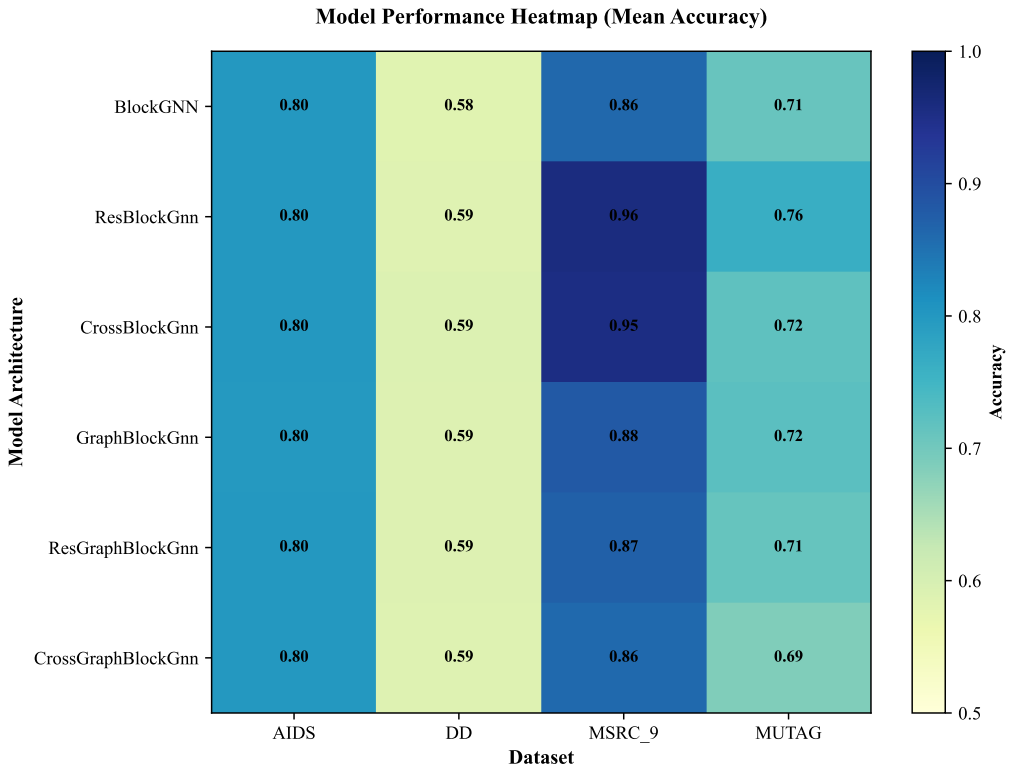


Figure 4: Model performance heatmap across datasets. Values indicate mean accuracy. MSRC_9 shows uniformly high performance, while DD presents the greatest challenge.

## 6.7 Discussion

### 6.7.1 Summary of Findings

Our experimental results lead to several key insights:

**Residual connections matter:** Both intra-branch (ResBlockGnn) and node-level cross-residual (CrossBlockGnn) mechanisms significantly improve performance over baseline sequential GNNs (BlockGNN). ResBlockGnn achieves the best average accuracy (0.776) and top performance on 2 out of 4 datasets.

**Depth tolerance is critical:** Residual mechanisms enable models to maintain performance at depth (h=4–5), whereas BlockGNN degrades severely. This validates that cross-residual connections mitigate over-smoothing, a fundamental challenge in deep GNNs.

**Cross-branch vs. intra-branch:** Interestingly, intra-branch residual (ResBlockGnn) outperforms node-level cross-residual (CrossBlockGnn) on average, suggesting that within-branch historical information is more critical than cross-branch exchange for these datasets. However, cross-branch models excel on specific datasets (e.g., MSRC_9), indicating their potential for multi-class problems.

**Efficiency trade-offs:** Cross-branch architectures impose significant computational overhead (55–177% slower). Practitioners should consider ResBlockGnn for most applications due to its strong performance-efficiency trade-off, while reserving CrossGraphBlockGnn for scenarios where maximal accuracy is required and computational resources are abundant.

### 6.7.2 Limitations

Our work has several limitations that suggest directions for future research:

1. **Computational overhead**: Cross-branch models are significantly slower than single-branch baselines, limiting their applicability to resource-constrained settings. Future work should explore more efficient cross-branch information exchange mechanisms (e.g., sparse cross-connections, gradient compression).

2. **Limited operator diversity**: Our experiments use only three operators (GCN, GAT, Transformer). Evaluating on a broader set of operators (e.g., GraphSAGE, GIN, MixHop) could further validate the framework's generality.

3. **Lack of statistical significance testing**: We report mean ± std but do not conduct paired t-tests or Wilcoxon signed-rank tests. Future work should include rigorous statistical analysis to confirm the significance of performance improvements.

4. **Dataset size constraints**: Experiments are conducted on relatively small datasets (188–2,000 graphs). Scalability to larger graph classification tasks (e.g., millions of graphs) remains untested.

5. **No edge attribute usage**: Current implementation does not leverage edge attributes, which could be important for datasets where edge types carry critical information (e.g., molecular bond types).

### 6.7.3 Conclusion

We presented ECR-GNN, a unified framework that integrates multiple graph operators through cross-branch residual connections. Through comprehensive experiments on four TUDataset benchmarks with 720 configurations, we demonstrated that:

1. ResBlockGnn achieves the best average accuracy (0.776) across all datasets

2. Residual mechanisms provide robust depth tolerance, maintaining performance at h=5 where BlockGNN degrades by 13.2%

3. Cross-branch models excel on multi-class problems (MSRC_9: 0.953–0.982)

4. Computational overhead ranges from +3.1% (ResBlockGnn) to +176.8% (CrossGraph-BlockGnn)

Our findings suggest that node-level and graph-level cross-residual connections offer complementary benefits: ResBlockGnn provides the best performance-efficiency trade-off for most applications, while CrossGraphBlockGnn may be preferred when maximal accuracy is required on larger datasets. The modular design of our framework enables seamless integration of new operators and residual mechanisms, facilitating future research on cross-branch GNN architectures.

# 7 Conclusion

This paper presented Enhanced Cross Residual Graph Neural Networks (ECR-GNN), a unified framework that integrates multiple graph convolution operators through structured cross-branch residual connections for improved graph classification. Unlike traditional approaches that treat different operators (GCN, GAT, Transformer) as alternative choices, ECR-GNN designs them as modular components that can be systematically combined through node-level and graph-level cross-residual mechanisms.

## 7.1 Summary of Contributions

Our key contributions are:

1. **Unified Cross-Branch Framework:** We proposed ECR-GNN, a novel architecture that unifies multiple graph operators (GCN, GAT, Transformer) through cross-branch residual connections, enabling operators with complementary inductive biases to collaborate synergistically rather than serving as independent alternatives.

2. **Dual Cross-Residual Mechanisms:** We designed two complementary cross-residual mechanisms: (1) node-level cross-residual connections that exchange intermediate representations between parallel branches at each layer (Eq. 5–6), and (2) graph-level cross-residual propagation that passes graph-level embeddings across branch pairs (Eq. 7–8), facilitating robust information flow across operators and depth scales.

3. **Modular Operator Instantiation:** We implemented a factory-based operator instantiation framework that treats graph convolution operators as pluggable components, enabling systematic evaluation of operator combinations and facilitating future integration of new operators (e.g., GraphSAGE, GIN) without modifying core cross-residual logic.

4. **Comprehensive Empirical Evaluation:** We conducted extensive experiments on four TUDataset benchmarks (MUTAG, DD, MSRC_9, AIDS) with 720 total configurations, evaluating six model architectures across three operators, five depths, and two embedding dimensions, providing rigorous analysis of performance-efficiency trade-offs.

## 7.2 Key Findings

Our experimental results reveal several important insights:

**ResBlockGnn achieves the best average performance:** Among all six model variants, ResBlockGnn (single-branch residual connections) ranks first overall with mean accuracy 0.776 across all datasets, achieving top performance on 2 out of 4 datasets (MUTAG: 0.832, MSRC_9: 0.982). This suggests that intra-branch residual connections alone provide significant benefits for graph classification.

**Residual mechanisms enable depth tolerance:** Node-level residual connections (ResBlockGnn, CrossBlockGnn) maintain performance at deep layers (h=4–5), whereas BlockGNN degrades severely by 13.2% from h=1 to h=5 on MUTAG. This validates that cross-residual

connections mitigate over-smoothing—a fundamental challenge in deep GNNs—by preserving multi-scale features and maintaining diverse receptive fields.

**Cross-branch architectures excel on specific datasets:** Cross-branch models (CrossBlockGnn, CrossGraphBlockGnn) demonstrate strong performance on MSRC_9 (0.953–0.982), a 9-class dataset with diverse graph structures, indicating their potential for multi-class problems where different operators can capture complementary structural patterns.

**Performance-efficiency trade-offs:** Cross-branch models impose significant computational overhead: CrossBlockGnn adds +55.7% execution time, while CrossGraphBlockGnn adds +176.8% time due to dual-branch parallel processing and sequential branch-pair computation. In contrast, ResBlockGnn offers the best trade-off with +2.4% average accuracy improvement over BlockGNN and only +3.1% time overhead.

### 7.3 Limitations

Our work has several limitations that suggest directions for future research:

1. **Computational overhead:** Cross-branch architectures are 55–177% slower than single-branch baselines, limiting their applicability to resource-constrained settings or large-scale graph classification tasks (millions of graphs). Future work should explore more efficient cross-branch information exchange mechanisms, such as sparse cross-connections activated only when necessary, or gradient compression techniques to reduce communication overhead.

2. **Limited operator diversity:** Our experiments evaluated only three operators (GCN, GAT, Transformer). While the modular framework supports easy integration of new operators, the generalizability of our findings to a broader set of operators (e.g., GraphSAGE, GIN, MixHop, FiLM) remains untested. Systematic evaluation on a wider operator library would further validate the framework's versatility.

3. **Lack of statistical significance testing:** We report mean accuracy ± standard deviation across 5-fold cross-validation but do not conduct rigorous statistical significance tests (e.g., paired t-tests, Wilcoxon signed-rank tests). Without such analysis, we cannot definitively claim that performance improvements are statistically significant rather than due to random variation. Future work should include statistical analysis to confirm the significance of observed improvements.

4. **Dataset size constraints:** Experiments are conducted on relatively small datasets (188–2,000 graphs). Scalability to larger graph classification benchmarks (e.g., millions of graphs) remains untested. The computational overhead of cross-branch architectures may become prohibitive at scale, requiring architectural optimizations or distributed training strategies.

5. **No edge attribute usage:** Current implementation does not leverage edge attributes, which could be critical for datasets where edge types or weights carry essential information (e.g., molecular bond types, social network tie strengths). Extending the cross-

residual framework to incorporate edge-aware message passing (e.g., relational GNN, edge-conditioned convolutions) is an important direction.

6. **Fixed cross-branch topology:** Our implementation uses fixed cross-branch connection patterns (e.g., branch 1 always exchanges with branch 2). Dynamic or learnable connection patterns that adapt during training—for example, using attention to determine which branches should communicate—could further improve performance by focusing information exchange on the most beneficial operator pairs.

### 7.4 Future Directions

Based on our findings and limitations, we identify several promising directions for future research:

**Efficient cross-branch mechanisms:** Explore sparse cross-connections, gradient compression, or knowledge distillation to reduce the computational overhead of cross-branch architectures while preserving their performance benefits. For example, instead of exchanging information at every layer, branches could communicate only every $k$ layers, or only when a learned gating mechanism detects insufficient information flow.

**Automated operator selection:** Develop neural architecture search (NAS) techniques to automatically determine the optimal operator combinations and cross-residual topologies for specific datasets. Instead of manually designing cross-branch architectures, an NAS controller could search the space of operator combinations (GCN, GAT, Transformer), residual mechanisms (intra-branch, node-level cross, graph-level cross), and hyperparameters (depth, width, dropout) to discover architectures optimized for given graph classification tasks.

**Theoretical analysis:** Conduct deeper theoretical analysis of why cross-branch residual connections mitigate over-smoothing and how information propagates through the architecture. For instance, can we derive bounds on representation diversity as a function of network depth and cross-branch connectivity? What are the spectral properties of cross-branch aggregation, and how do they relate to graph signal processing theory? Such theoretical understanding could guide the design of more effective cross-branch architectures.

**Extension to other tasks:** Adapt the cross-residual framework to node classification, link prediction, and graph generation tasks. For node classification, graph-level cross-residual connections could be replaced by node-level cross-branch exchanges that operate on intermediate node representations. For graph generation, cross-branch mechanisms could enable different decoder branches to share information during autoregressive generation.

**Integration with other advances:** Combine ECR-GNN principles with other advances in graph neural networks, such as graph transformers (which apply self-attention to all node pairs), equivariant GNNs (which respect geometric symmetries), or message-passing neural networks with learned aggregation functions. For example, a graph transformer could serve as one branch, while a GCN serves as another, with cross-residual connections enabling the transformer to benefit from the GCN's inductive bias for local structure.

**Real-world applications:** Apply ECR-GNN to practical domains to validate its effectiveness beyond academic benchmarks. Potential applications include molecular property prediction (combining GCN for bond topology with GAT for atom-atom interactions), drug discovery

(multi-task prediction where different operators specialize on different molecular properties), social network analysis (leveraging both smoothing and selective attention patterns), and recommendation systems (integrating collaborative filtering with content-based filtering through cross-branch information exchange).

### 7.5 Closing Remarks

Graph classification remains a fundamental and challenging task with numerous real-world applications in bioinformatics, cheminformatics, social network analysis, and computer vision. The ECR-GNN framework presented in this paper provides a novel perspective by unifying multiple graph operators through cross-branch residual connections, enabling them to collaborate synergistically rather than serving as independent alternatives.

Our experiments demonstrate that cross-residual mechanisms—particularly node-level residual connections—provide robust depth tolerance and consistent performance improvements across diverse datasets. However, the computational overhead of cross-branch architectures represents a significant trade-off that practitioners must weigh against accuracy gains. ResBlockGnn, with its minimal overhead (+3.1% time) and strong performance (+2.4% accuracy), emerges as a practical choice for most applications, while CrossGraphBlockGnn may be reserved for scenarios where maximal accuracy is required and computational resources are abundant.

We believe this work will inspire further research into cross-branch GNN architectures, modular operator composition, and efficient information exchange mechanisms. The open-source implementation based on PyTorch Geometric facilitates reproducibility and enables the research community to build upon our work, exploring new operators, residual mechanisms, and applications. By providing a unified framework that treats graph convolution operators as composable components rather than monolithic choices, we hope to accelerate the development of more expressive, scalable, and effective graph neural networks for graph classification and beyond.

## References


[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in neural information processing systems*, 2017, pp. 1024–1034.

[4] V. P. Dwivedi and X. Bresson, "A generalization of transformer networks to graphs," *arXiv preprint arXiv:2012.09699*, 2020.

[5] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Advances in neural information processing systems*, 2018, pp. 4800–4810.

[6] J. Lee, I. Lee, and J. Kang, "Self-attention graph pooling," *arXiv preprint arXiv:1904.08082*, 2019.

[7] G. Li, E. Muller, A. Thabet, and B. Ghanem, "Deeper insights into graph convolutional networks for semi-supervised learning," *arXiv preprint arXiv:1809.02584*, 2018.

[8] Y. Chen, X. Wei, P. Xu, X. Wang, P. Luo, Z. Li, and J. Tang, "Revisiting over-smoothing in deep gcns," *arXiv preprint arXiv:2003.13663*, 2020.

[9] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations (ICLR)*, 2019.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[11] X. Bresson and T. Laurent, "Residual gated graph convnets," *arXiv preprint arXiv:1711.07553*, 2017.

[12] L. Zhao and L. Akoglu, "Pairnorm: Tackling over-smoothing in gns," *arXiv preprint arXiv:2009.10698*, 2020.

[13] T. Cai, J. Luo, S. Wang, K. Zhang, Y. Tian, L. Yang, Z. Li, and K. Weinberger, "Graphnorm: A principled approach to accelerating graph neural network training," in *International Conference on Machine Learning*, 2021, pp. 1277–1287.

[14] Y. Rong, W. Huang, T. Xu, and J. Huang, "Dropedge: Towards deep graph convolutional networks on node classification," *arXiv preprint arXiv:1907.10903*, 2019.

[15] K. Xu, C. Li, Y. Tian, X. Du, J. Leskovec, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *International Conference on Machine Learning*, 2018, pp. 5449–5458.

[16] H. Du, Y. Zhang, P. Zhang, B. Gu, X. Wei, J. Guo, A. Li, Z. Wang, T. Ma *et al.*, "Densegnn: universal and scalable deeper graph neural networks for high-performance property prediction in crystals and molecules," *npj Computational Materials*, vol. 10, no. 1, p. 304, 2024.

[17] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, G. Steeg, and A. Galstyan, "Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing," in *International Conference on Machine Learning*, 2019, pp. 33–42.

[18] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," *arXiv preprint arXiv:1903.02428*, 2019.

[19] M. Gori, G. Monfardini, and F. Scarselli, "A new model for learning in graph domains," *Proceedings of IJCNN*, 2005.

[20] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *International Conference on Machine Learning*, 2017, pp. 1263–1272.

[21] D. K. Duvenaud, D. Maclaurin, J. Iparraguirre, R. Bombarell, T. Hirzel, A. Aspuru-Guzik, and R. P. Adams, "Convolutional networks on graphs for learning molecular fingerprints," in *Advances in neural information processing systems*, 2015, pp. 2224–2232.

[22] C. Cangea, P. Veličković, N. Jovanović, T. Kipf, and P. Liò, "Towards sparse hierarchical graph classifiers," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2018, pp. 165–176.

[23] C. Liu, X. Wang, Y. Li, Y. Wang, and J. Gao, "Graph pooling for graph neural networks," pp. 4254–4261, 2023.

[24] Z. Li, H. Chen, Z. Zeng, J. He, X. Mao, N. Lv, and L. Chen, "Graph pooling for graph-level representation learning: a survey," *Artificial Intelligence Review*, vol. 57, no. 8, pp. 1–51, 2024.

[25] M. Zhang, Y. Liu, Q. Chen, and S. Pan, "Personalized graph neural networks with attention mechanism," 2019.

[26] D. Beaini, S. Passaro, E. Létouzey, J. Domke, S. Erdos, P. Lió, P. Veličković, and M. M. Bronstein, "Directional graph networks," *arXiv preprint arXiv:2010.02863*, 2020.

[27] E. Rossi, F. Frasca, F. M. Bianchi, N. Alletto, and P. Lio, "Edge directionality improves learning on heterophilic graphs," in *International Conference on Learning Representations (ICLR)*, 2023.

[28] F. M. Bianchi, F. Grattarola, and C. Alippi, "Graph neural networks with convolutional arma filters," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

[29] A. Gravina, N. Alletto, A. Rossi, F. M. Bianchi, and P. Lio, "Anti-symmetric dgn: A stable architecture for deep graph networks," *arXiv preprint arXiv:2210.09789*, 2022.

[30] S. Li, M. Zhang, M. Jin, H. Zhang, R. Ying, Z. Li, and J. Yan, "A non-asymptotic analysis of oversmoothing in graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2023.

[31] J. Topping, F. Di Giovanni, B. Chamberlain, M. Bronstein, D. Vendrig, and P. Lio, "Understanding over-squashing and bottlenecks on graphs via curvature," *arXiv preprint arXiv:2111.14522*, 2021.

[32] U. Alon and E. Yahav, "On the bottleneck of graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2021.

[33] Y. Li, X. Wang, Z. Wang, and J. Li, "Learning graph normalization for graph neural networks," *Neurocomputing*, vol. 486, pp. 140–151, 2022.

[34] G. Li, E. Müller, A. Thabet, and B. Ghanem, "Training graph neural networks with 1000 layers," in *International Conference on Machine Learning*, 2021, pp. 6403–6413.

[35] K. Oono and T. Suzuki, "Simple and deep graph convolutional networks," *arXiv preprint arXiv:2006.13604*, 2020.

[36] G. Wang, I. Balazevic, M. Znis, M. Gitta, Y. Kwon, M. Seeger, X. Li, L. De Raedt, and M. Welling, "Multi-hop attention graph neural networks," in *International Joint Conference on Artificial Intelligence*, 2021, pp. 2112–2120.

[37] X. Sun, R. Wang, L. Cui, X. Kong, and S. Li, "Attention-based graph neural networks: a survey," *Artificial Intelligence Review*, vol. 56, no. 11, pp. 13 493–13 539, 2023.

[38] J. Cai, X. Zheng, C. Luo, C. Yang, and E. Santus, "Understanding graph embedding methods and their applications," *arXiv preprint arXiv:2012.08019*, 2020.

[39] H. Van Pham, U. The, T. Do, and A. Nguyen, "Hierarchical pooling in graph neural networks to enhance graph classification," *Applied Sciences*, vol. 11, no. 17, p. 8164, 2021.

[40] H. Gao and S. Ji, "Graph u-net," *arXiv preprint arXiv:1905.05178*, 2019.

[41] J. Klicpera, A. Bojchevski, and S. Günnemann, "Combining label propagation and simple models out-performs graph neural networks," *arXiv preprint arXiv:1810.05997*, 2018.

[42] L. Zhao, Z. Leng, Y. Chen, and L. Akoglu, "Tackling over-smoothing for general graph learning," *arXiv preprint arXiv:2008.09864*, 2020.

[43] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[44] N. Keriven, G. Peyré, and S. Vaiter, "A unified benchmark for the diagnosis and treatment of graph neural networks," *arXiv preprint arXiv:2012.01450*, 2020.